- **Configure git (first time).**

git config --global user.name "Your Name"
git config --global user.email your_email@example.com ---- its a terminal access.

- **To check the current origin.**

git remote –v
Example showing like this.
origin https://github.com/username/my-first-repo.git (fetch)
origin https://github.com/username/my-first-repo.git (push)

- **Login to git-hub**

To create a new repository.
Name as you like suitable for your project name. (e.g shankar.repo),
You can choose public or private (visible for all or personal can use).
Create repository.

- **you have three main options for the remote URL**
  HTTPS
  SSH
  GitHub CLI

1. HTTPS:

HTTPS (HyperText Transfer Protocol Secure) is the standard, When you clone using HTTPS, your computer communicates with GitHub via a secure protocol (https://).

Clone the code via this url through

git clone https://github.com/username/repository-name.git

2.SSH:

SSH (Secure Shell) is a more secure interact with GitHub repositories.

Generate an **SSH key** on your local machine.

ssh-keygen -t rsa -b 4096 -C "your_email@example.com" Use this command.

The public key is added to your GitHub account, and the private key remains on your computer.

git clone git@github.com:username/repository-name.git

3. GitHub CLI (But its i am not use)

GitHub CLI (Command-Line Interface)

gh auth login

gh repo clone username/repository-name (an all-in-one tool for managing GitHub tasks.)

One open terminal: -

Git init

Git branch ------ showing your repo branch details.

Git checkout –b (feature/xyz)   To create switch new branch.

Git status (Showing modified files list)

Git add . (stage all changes)

Git commit –m “add something like”
git push origin feature/xyz **(Sub-branch)**
git branch -d feature-branch

Merging Changes via Pull Request (PR)

Goto repo on github,
Click **"Compare & pull request"**.
Add a description and assign reviewers.

Review & Merge

- Team members review the code.
- If approved, the PR is merged into main.
- Resolve conflicts (if any) before merging.

Resolution Flowchart:

Identify conflicts → 2. Edit files → 3. Mark resolved → 4. Commit

Git Versioning:

Versioning helps track stable points in your project's history using **tags**. GitHub **releases** package these tags with release notes and downloadable assets.

git status
git add .
git tag -a v1.0.0 -m "First release version 1.0.0" ------------- creating a tag and pushing
git tag
git push origin v1.0.0
git push origin –tags
git tag -d v1.0 ------ delete tags.

- **previous changes remove in git.**

git revert (commit id) ------ delete previuos edit or added code remove.
git log --oneline ----- commits showing all
git revert 12a19c4 --no-edit ----- old file edit or delete
git log –oneline